Mini Project 1: Music Listening Logger — Notes & Code

C Language Study Notes & Comprehensive Guide

Project Concepts & Overview

Concepts Combined:

- **Arrays** — store 7 days of listening minutes.
- **Loops** — input data, display summaries, and repeat the menu.
- **File Handling** — save/load data for **persistence** (data survives after the program closes).
- **Menu-driven interface** — a `while` loop showing options until the user exits.

File Handling in This Project:

Mode	Purpose / Action
`"w"`	Reset / Write — clears `music_log.txt` and writes fresh data
`"a"`	Append — adds each new log entry (if applicable)
`"r"`	Read — opens saved data to build the weekly report

Design Tips:

- Always check `fopen()` return values for `NULL`.
- Confirm destructive actions (like resetting data) before executing them.
- Use modular functions to keep the menu loop clean (`logMinutes()`, `generateReport()`, etc.).

1 Task 1: Log Daily Listening Minutes into an Array

Notes: Declare a 1D array of size 7 and a 2D array of day labels. Use a `for` loop to capture daily user input via `scanf`.

```c
#include <stdio.h>

int main() {
    int minutes[7];     // one slot per day of the week
    char days[7][10] = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};

    // Input listening minutes for each day using a loop
    for (int i = 0; i < 7; i++) {
        printf("Enter minutes listened on %s: ", days[i]);
        scanf("%d", &minutes[i]);
    }

    // Display the entered data
    printf("\n----- Your Week -----\n");
    for (int i = 0; i < 7; i++) {
        printf("%s : %d minutes\n", days[i], minutes[i]);
    }

    return 0;
}
```

Sample Output:

```
Enter minutes listened on Mon: 30
Enter minutes listened on Tue: 45
...
----- Your Week -----
Mon : 30 minutes
Tue : 45 minutes
...
```

2 Task 2: Menu-Driven Interface

Notes: Wrap functionality inside functions and use an infinite `while (1)` loop with conditional checks to create an interactive CLI menu.

```c
#include <stdio.h>

int minutes[7] = {0, 0, 0, 0, 0, 0, 0};
char days[7][10] = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};

// Log listening minutes for the week
void logMinutes() {
    printf("\n--- Log Listening Minutes ---\n");
    for (int i = 0; i < 7; i++) {
        printf("Enter minutes for %s: ", days[i]);
        scanf("%d", &minutes[i]);
    }
    printf("Weekly data logged!\n");
}

// Show weekly summary
void showSummary() {
    int total = 0;
    printf("\n--- Weekly Summary ---\n");
    for (int i = 0; i < 7; i++) {
        printf("%s : %d minutes\n", days[i], minutes[i]);
        total += minutes[i];
    }
    printf("Total: %d minutes\n", total);
}

int main() {
    int choice;

    while (1) {
        printf("\n===== Music Listening Logger =====\n");
        printf("1. Log listening minutes\n");
        printf("2. View weekly summary\n");
        printf("3. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        if (choice == 1) {
            logMinutes();
        } else if (choice == 2) {
            showSummary();
        } else if (choice == 3) {
            printf("Goodbye! Keep listening \n");
            break;
        } else {
            printf("Invalid choice! Try again.\n");
        }
    }
    return 0;
}
```

3 Task 3: Save Logs to `music_log.txt` (Persistence)

Notes: Open file in `"w"` mode, check for `NULL`, and write each day's minutes using `fprintf` so data persists across executions.

```c
#include <stdio.h>

int minutes[7] = {0, 0, 0, 0, 0, 0, 0};
char days[7][10] = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};

void logMinutes() {
    FILE *fp;

    printf("\n--- Log Listening Minutes ---\n");
    for (int i = 0; i < 7; i++) {
        printf("Enter minutes for %s: ", days[i]);
        scanf("%d", &minutes[i]);
    }

    fp = fopen("music_log.txt", "w");
    if (fp == NULL) {
        printf("Error: could not open music_log.txt!\n");
        return;
    }
    for (int i = 0; i < 7; i++) {
        fprintf(fp, "%d\n", minutes[i]);
    }
    fclose(fp);

    printf("Data saved to music_log.txt!\n");
}

int main() {
    int choice;
    while (1) {
        printf("\n===== Music Listening Logger =====\n");
        printf("1. Log listening minutes\n");
        printf("2. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        if (choice == 1) logMinutes();
        else if (choice == 2) break;
    }
    return 0;
}
```

Sample `music_log.txt` File Contents:

```
30
45
60
20
90
120
75
```

4 Task 4: Weekly Report from File (Total, Average, Highest)

Notes: Read back data from `music_log.txt` using `fscanf`, calculate the sum, find the peak day using max tracking, and compute a float average.

```c
#include <stdio.h>

void generateReport() {
    FILE *fp;
    int minutes[7];
    int total = 0, highest = 0;
    float average;

    fp = fopen("music_log.txt", "r");
    if (fp == NULL) {
        printf("No saved data found! Log your minutes first.\n");
        return;
    }

    for (int i = 0; i < 7; i++) {
        fscanf(fp, "%d", &minutes[i]);
        total += minutes[i];
        if (minutes[i] > highest) {
            highest = minutes[i];
        }
    }
    fclose(fp);

    average = (float)total / 7;

    printf("\n====== Weekly Listening Report ======\n");
    printf("Total listening time   : %d minutes\n", total);
    printf("Average per day        : %.2f minutes\n", average);
    printf("Highest listening time : %d minutes\n", highest);
}

int main() {
    generateReport();
    return 0;
}
```

Sample Output:

```
====== Weekly Listening Report ======
Total listening time : 440 minutes
Average per day : 62.86 minutes
Highest listening time : 120 minutes
```

5 Task 5: Complete Program with Reset Confirmation

Notes: Combines logging, reporting, file persistence, and data reset with user confirmation (`y/n`) into a robust menu-driven application.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int minutes[7] = {0, 0, 0, 0, 0, 0, 0};
char days[7][10] = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};

void logMinutes() {
    FILE *fp = fopen("music_log.txt", "w");
    printf("\n--- Log Listening Minutes ---\n");
    for (int i = 0; i < 7; i++) {
        printf("Enter minutes for %s: ", days[i]);
        scanf("%d", &minutes[i]);
    }
    if (fp == NULL) {
        printf("Error: could not save data!\n");
        return;
    }
    for (int i = 0; i < 7; i++) {
        fprintf(fp, "%d\n", minutes[i]);
    }
    fclose(fp);
    printf("Data saved to music_log.txt!\n");
}

void generateReport() {
    FILE *fp = fopen("music_log.txt", "r");
    int total = 0, highest = 0;
    float average;

    if (fp == NULL) {
        printf("\nNo saved data found! Log your minutes first.\n");
        return;
    }

    printf("\n--- Your Week ---\n");
    for (int i = 0; i < 7; i++) {
        fscanf(fp, "%d", &minutes[i]);
        printf("%s : %d minutes\n", days[i], minutes[i]);
        total += minutes[i];
        if (minutes[i] > highest) highest = minutes[i];
    }
    fclose(fp);

    average = (float)total / 7;
    printf("\n====== Weekly Report ======\n");
    printf("Total   : %d minutes\n", total);
    printf("Average : %.2f minutes/day\n", average);
    printf("Highest : %d minutes\n", highest);
}

void resetData() {
    char confirm;
```

```
    printf("\nAre you sure you want to delete ALL weekly data? (y/n): ");
    scanf(" %c", &confirm);

    if (confirm == 'y' || confirm == 'Y') {
        memset(minutes, 0, sizeof(minutes));
        FILE *fp = fopen("music_log.txt", "w");
        if (fp != NULL) fclose(fp);
        printf("Weekly data has been reset!\n");
    } else {
        printf("Reset cancelled. Your data is safe.\n");
    }
}

int main() {
    int choice;
    printf("Welcome to Music Listening Logger \n");

    while (1) {
        printf("\n===== MAIN MENU =====\n");
        printf("1. Log listening minutes\n");
        printf("2. View weekly report\n");
        printf("3. Reset weekly data\n");
        printf("4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: logMinutes();    break;
            case 2: generateReport(); break;
            case 3: resetData();      break;
            case 4:
                printf("Goodbye! Keep listening \n");
                exit(0);
            default:
                printf("Invalid choice! Try again.\n");
        }
    }
    return 0;
}
```

6 Quick Revision Summary

Component	Mechanism / Syntax	Purpose
Arrays	`int minutes[7]`	Hold 7 days of listening stats
Menu Loop	`while (1) + switch`	Keep app running until user exits
Persistence	`fopen("file.txt", "w"/"r")`	Save data to disk & load later
Report Math	Sum, Average, Max	Analyze weekly listening habits
Reset Safety	`memset` + confirmation	Secure deletion with (y/n) prompt

Golden Rules of Mini Project Building:

- Modularize your code into distinct functions for each menu option.
- Always validate file pointers (`fp == NULL`) before reading or writing.
- Provide safe confirmation prompts for destructive operations.